

JAVA · MAVEN · COMMAND-LINE TOOL

AI-Based Resume Analyzer

A rule-based Java tool that scores how well a resume matches a job description — and tells you exactly which skills are missing.

Name: Akshat Bajpai

Registration No.: 25BAI10500

Slot: A21+A23



THE PROBLEM

Matching resumes to jobs is still mostly manual guesswork

A candidate reads a job description, skims their own resume, and tries to guess how good a fit they are. There's no consistent way to see which specific skills are already covered — and which ones are actually missing.

1

Inconsistent

Every read-through gives a slightly different impression of “fit.”

2

No gap visibility

Missing skills stay hidden until an interviewer points them out.

3

Hard to act on

“Improve your resume” isn't useful without knowing what to fix.

RESUME

Java

Python

SQL

Git

JOB DESCRIPTION

Java

Spring Boot

MySQL

REST API

Six steps from raw text to an actionable report

Ti

Reads plain text

Loads a resume and a job description from .txt files
— no special format needed.

Sk

Detects skills

Scans both documents against a dictionary of ~38
common technical skills.

Cp

Compares the two

Finds which required skills already appear on the
resume.

%

Scores the match

Calculates a match percentage from required vs.
matched skills.

Gp

Flags the gaps

Lists exactly which required skills are missing from
the resume.

Rp

Saves a report

Writes a full analysis report to output/analysis-
report.txt.

One straight pipeline, five stages



Every stage is a small, single-purpose Java class — easy to trace and easy to test.

```
$ java -jar target/ai-based-resume-analyzer-1.0.0.jar  
→ output/analysis-report.txt
```

Eight small classes, each with one job

01

Main

CLI entry point — wires everything together

02

ResumeParser

Reads .txt files, extracts name/title & skills via regex

03

Resume

Holds a candidate's name, raw text & skill set

04

JobDescription

Holds a job's title, raw text & required skills

05

SkillMatcher

Compares two skill sets, computes the score

06

ResumeAnalyzer

Coordinates parsing + matching into one result

07

AnalysisResult

Immutable record of the full analysis outcome

08

ReportGenerator

Formats the result as text, prints & saves it

Simple math, transparent logic

No black box — the score is a plain ratio, so anyone reading the code can verify exactly how it was calculated.

$$\text{Match \%} = \frac{\text{Matched Required Skills}}{\text{Total Required Skills}} \times 100$$

Skills come from a fixed dictionary — languages, frameworks, tools and a couple of soft skills — matched with word-boundary-aware regex so “Java” doesn't match inside “JavaScript.”



Matched skill

Appears in both the resume and the required-skills list.

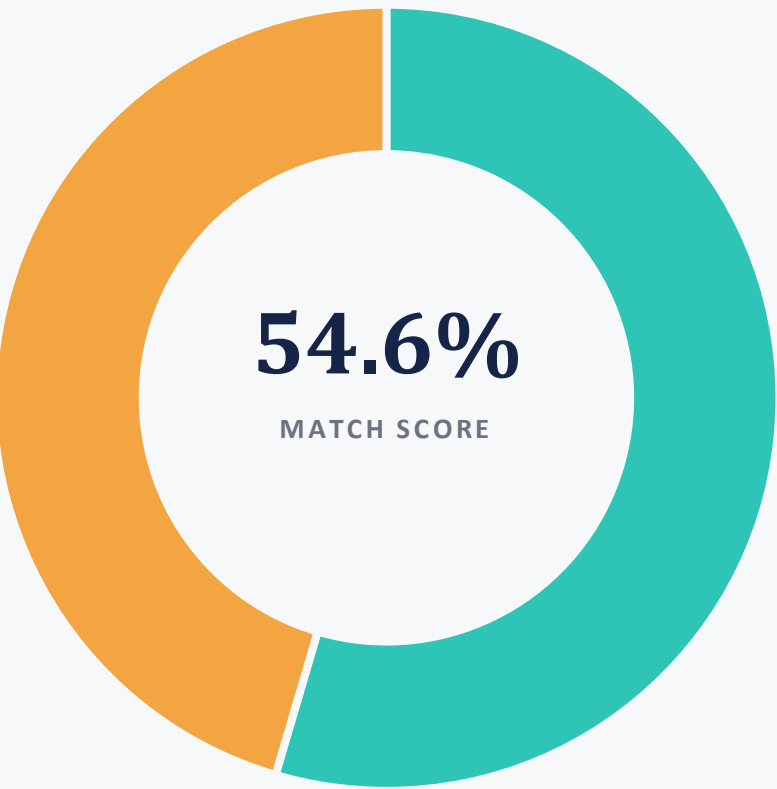


Missing skill

Required by the job, not detected in the resume.

Run against the sample files

Akshat Bajpai's sample resume vs. the sample "Java Developer" job description, shipped with the project.



6 of 11 required skills detected

MATCHED

- ✓ Java
- ✓ SQL
- ✓ Git
- ✓ GitHub
- ✓ Data Structures
- ✓ Algorithms

MISSING

- ! Spring Boot
- ! Spring
- ! REST API
- ! API
- ! MySQL

SAMPLE OUTPUT

Printed to the console — and saved to disk

analysis-report.txt

=====

RESUME ANALYSIS REPORT

=====

Candidate : Akshat Bajpai

Job Title : Java Developer

Matched Skills

• Java • SQL • Git • GitHub • Data Structures • Algorithms

Missing Skills

• Spring Boot • Spring • REST API • API • MySQL

Match Score

54.55%

Note: rule-based text analysis, not a trained ML model.

Built with a small, standard toolset

J

Java 17

Language & runtime — records, modern switch, text blocks

M

Maven

Build tool — dependency & lifecycle management

C

Collections

Sets, LinkedHashSet — skill storage & comparison

F

File I/O

java.nio.file — reading input, writing the report

R

Regex

Pattern/Matcher — word-boundary skill detection

C

CLI (Scanner)

Simple interactive prompts for file paths

“AI-Based” — but not a trained model

This project automates a task — skill extraction and gap analysis — that a person would otherwise do by hand. That's the sense in which it's “AI-based.”

It does not use a neural network, embeddings, or any trained model. Matching is done with a fixed skill dictionary and regular expressions — deterministic, transparent, and easy to audit.

What it IS

- Rule-based text analysis
- Fixed skill dictionary
- Deterministic & explainable

What it is NOT

- A trained ML / NLP model
- Semantic or contextual matching
- A replacement for human review

Where this could go from here

1

PDF / DOCX input

Parse real resume files instead of plain text

3

Web interface

Drag-and-drop upload instead of a CLI prompt

5

Persistent storage

Save analysis history to a database

2

NLP & semantic matching

TF-IDF or embeddings instead of exact keywords

4

Multi-job comparison

Score one resume against several postings at once

6

Guided suggestions

Concrete, personalized resume-improvement tips

